SKALA AI 서비스 MINI-PROJECT · 과제 1번

Base AI Agent의 RunBook 선택 개선

RAG 기반 RunBook 선택 · 라우팅 기준 확대 · 데이터 확장 · 평가 하네스

2분 — 기능과 결과
무엇을 고쳤나 · Before/After 수치

3분 — 코드 설명
Agent Graph 구성 · MCP · RAG

79번 박건우 (G079 · 광주 3반) · 2026-10-21

FastAPI + LangGraph v2 + MCP

all_total_vue_student

github.com/pigpgw/skala-ai-service-mini

2분 개선한 기능

판정 하나를 바꿨고, 그걸 측정할 수 있게 만들었다

기존 판정은 키워드 규칙을 순서대로 보다가 **처음 맞는 것**을 고르는 방식이었다. 규칙 순서와 증상 우선순위가 무관하고, 질의문은 보지 않았다.

축	바꾼 것	결과
1 RAG 판정	키워드 첫 매칭 → 런북 임베딩 의미 검색 (확신 없으면 keyword 폴백)	0.3250 → **0.8500**
2 라우팅	결정론 백엔드의 라우팅 정규식이 7개 키워드뿐 → 운영 어휘로 확대	회귀 3/10 → **10/10**
3 데이터	모니터링 6→12시스템 · 런북 6→10종 · 파일 선택 한 곳으로 통합	unknown 18.75% → **0%**
4 평가	실행 1회 = 폴더 1개 하네스 · 테스트 65개 · 회귀 실행기	재현성 **identical**

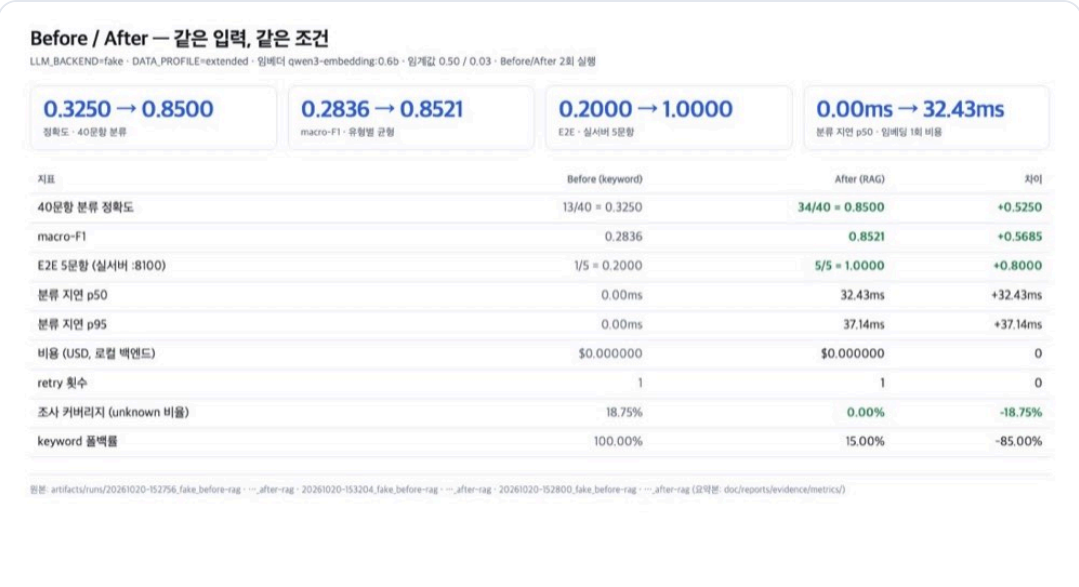
기준선을 먼저 재고 → 어디가 틀렸는지 코드 위치로 짚은 뒤 → 고치고 → 같은 조건 2회 실행으로 비교했다.

2분 결과

같은 입력, 같은 조건 — 2회 실행한 결과

지표	Before	After	차이
40문항 정확도	0.3250	0.8500	+0.5250
macro-F1	0.2836	0.8521	+0.5685
E2E 5문항(실서버)	0.2000	1.0000	+0.8000
회귀 질의 10문항	3/10	10/10	+0.7000
분류 지연 p50	0.0ms	32.43ms	+32.43ms
비용 / retry	$0 / 1	$0 / 1	0 / 0

한계도 그대로: 데이터 확장은 정확도를 0 올렸습니다 (base 34/40 ≡ extended 34/40). margin 게이트는 2문항을 아꼈습니다.
keyword 폴백률은 15.00% — 40문항 중 6건만 keyword로 내려갔고, 그 6건은 1·2위 유사도 차이가 0.0155·0.0000인 동점이었습니다.

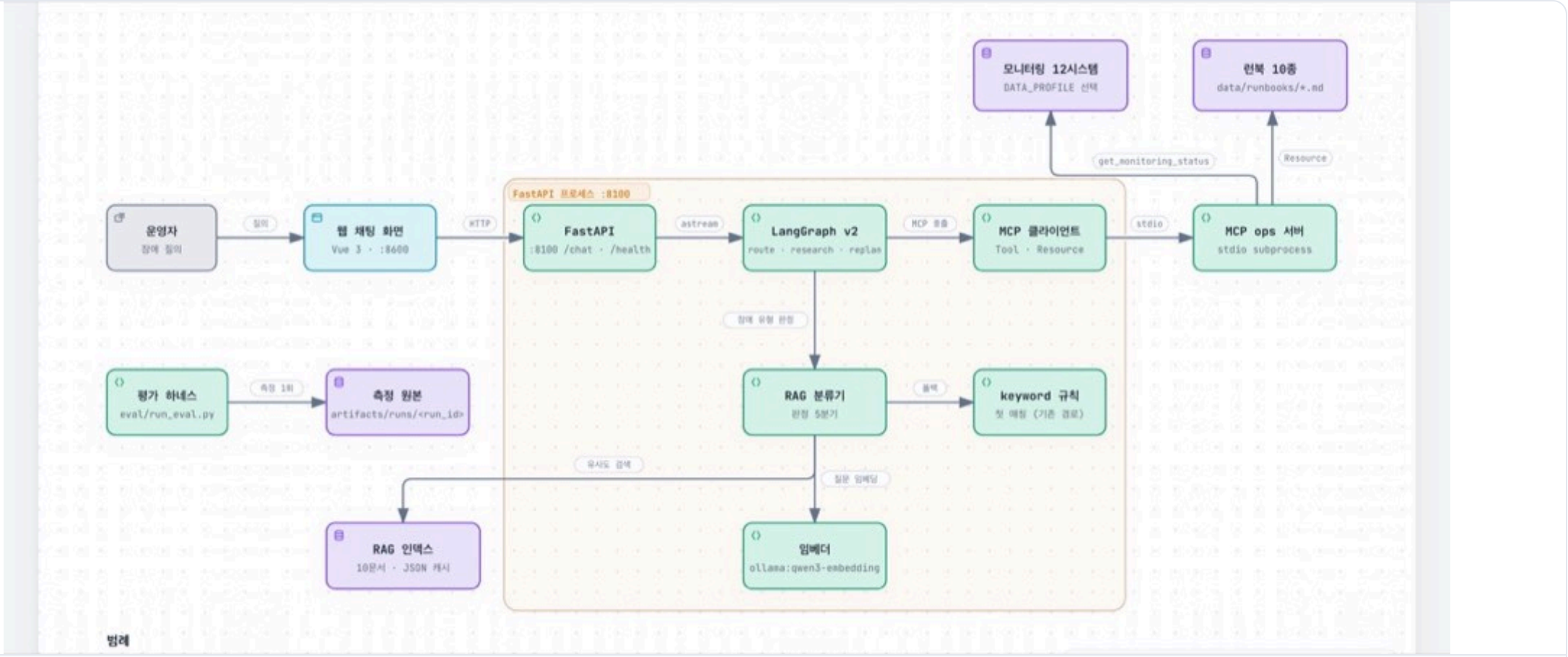


Before/After 수치 화면 (원본: artifacts/runs/ → doc/reports/evidence/metrics/)

3분 Agent Graph 구성

LangGraph v2 — 판정 지점만 갈아끼운 그래프

route → plan → gate → research → **replan_check** → analysis → write → validate → finalize. `replan_check`가 "모onitoring 정보만으로는 부족해 런북이 필요하다"고 판단하는 지점이고, 그 판단을 RAG로 바꿨다.



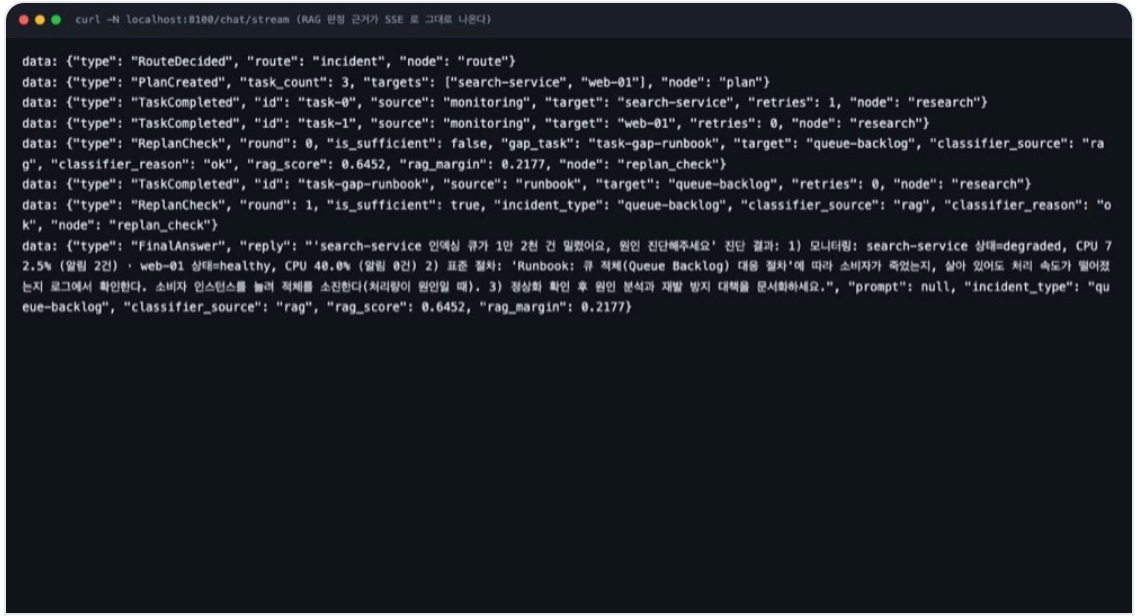
FastAPI 프로세스(:8100) 안의 그래프와, MCP ops 서버(stdio subprocess)·RAG 인덱스·평가 하네스

3분 RAG 코드

런북 임베딩 검색 + 확신 없으면 기존 경로로

```
// agent/rag/classifier.py
if not self.enabled:              → keyword
if index is None:                 → keyword
vector = embedder.embed([text])[0]     # [질문][시스템][상태][알림]
top1, top2 = index.search(vector)
if top1 < min_score:      → keyword  (0.50)
if top1-top2 < min_margin: → keyword  (0.03)
return top1                       # rag
```

- `agent/rag/embedder.py` — Ollama `qwen3-embedding:0.6b`
- `agent/rag/index.py` — 10문서 인덱스 + JSON 캐시
- 선택 근거는 `/health`·`metadata`·SSE로 노출



실제 SSE 출력 — classifier_source=rag, rag_score=0.6452, rag_margin=0.2177

3분 MCP 코드

MCP로 모니터링·런북을 조회하고, 데이터 선택은 한 곳에서

```
# mcp_server.py (ops 서버)
@mcp.tool
def get_monitoring_status(system_name) → SystemStatus
@mcp.resource
def get_incident_runbook(incident_type) → str
@mcp.prompt
def incident_analysis_prompt(system_name, symptom)
```

```
# agent/data_profile.py
DATA_PROFILE=base|extended
→ 파일 선택의 단일 결정 지점 (하드코딩 5곳 제거)
```

- `agent/mcp_client.py` — stdio subprocess로 ops 서버 연결, `find_resource_template`로 런북 URI 탐색
- `mcp_server_docs.py`는 decoy — 의도대로 쓰지 않는다
- 파일 선택이 5곳에 흩어져 있으면 시스템만 늘고 조사 대상은 6개로 잘린다 → 한 곳으로 모았다

의존성 주입: `SupervisorDeps`에 `rag`을 넣고 lifespan에서 1회 생성 → 노드가 `os.environ`을 직접 읽지 않는다.

LangSmith: 노드에 `@traceable` 적용, `.env`로 트레이싱 켜짐 — 실 API 키 없는 환경이라 캡처는 미수행

마무리

판정을 바꾼 게 아니라,
판단의 근거를 남기고 측정할 수 있게 바꾼 것

배운 점

- RAG는 임베딩보다 **입력 정의**와 **게이트**가 핵심이다
- 개선 전에 기준선을 먼저 재야 "개선"을 말할 수 있다
- 근거를 남기면 질문이 와도 바로 보여줄 수 있다

다음에 하고 싶은 것

- 실제 장애 티켓으로 라벨 세트 재구축 (지금은 자기 평가)
- LLM-as-judge 평가 추가 · Research 서브그래프 분리
- 실 OpenAI 백엔드로 비용·지연 재측정

40문항 **0.3250** → **0.8500** · macro-F1 **0.2836** → **0.8521** · 회귀 **3/10** → **10/10** · 테스트 **65개** · 같은 조건 2회 실행 결과 동일